ZOZAPRIME: STRATEGIC REALIGNMENT & TECHNICAL DEVELOPMENT SPRINT

To: Emmanuel Erogo
From: Guandaru Dedan Kimathi
Date: May 18, 2026
Subject: Operational Restructuring, Founder Agreements & Critical Tech Roadmap

---

Emmanuel,

Since tonight's sync couldn't happen, I've consolidated our strategic position, operational bottlenecks, and the immediate technical roadmap that will take us from prototype to production-grade infrastructure. This document establishes clear responsibilities, formalizes our capital structure, and outlines your development mandates for Q2 2026.

---

1. FOUNDER ROLES & ASYNCHRONOUS EXECUTION MODEL

1.1 Geographic & Operational Reality

Your Position: Taita Taveta, balancing Red Cross engineering commitments alongside ZOZAPRIME development
My Position: Nairobi, handling frontline commercial negotiations, institutional partnerships, and strategic positioning

Decision: We are transitioning to a model to eliminate coordination friction and maximize parallel execution velocity.

1.2 Division of Custody & Accountability

Domain	Owner	Scope
Engineering & Repository	Emmanuel Erogo	100% custody of codebase execution, system stability, API development, deployment pipeline, sprint deadlines
Commercial & Institutional	Guandaru Kimathi	100% custody of negotiations,event partnerships , investor outreach, grant applications etc
Quality Assurance (QA)	Guandaru Kimathi	Final acceptance testing, user flow validation, pre-launch stress testing

Domain	Owner	Scope
Product Strategy	Joint (async documentation)	Feature prioritization via GitHub Projects, technical specifications via Notion/Linear

Communication Protocol:

- **Async-First:** All strategic decisions documented in Notion, technical specs in GitHub Issues
- **Sync Calls:** Weekly 1-hour sprint review (Sundays 8pm preferred)
- **Blockers:** Flagged immediately via WhatsApp with 24-hour response SLA

---

2. CAPITAL GOVERNANCE & FOUNDER EQUITY STRUCTURE

2.1 Current Legal Standpoint
Legal Status: Currently operating as General Partnership, transitioning to **Private Limited Company (LLC)** by Q3 2026

2.2 Formalization Framework

Objective: Prepare for $50,000 seed/grant round by December 2026

Immediate Actions:

1. **Capital Account Ledger**
    - All historical personal capital injections will be logged with supporting evidence (M-Pesa statements, receipts, bank transfers)
    - Structure: Date | Amount | Source (Personal Savings / Loan / Grant) | Purpose | Receipt Reference
    - **Deadline:** May 31, 2026
2. **LLC Incorporation Timeline**
    - **June 2026:** Engage corporate attorney (Technopolis Law or equivalent)
    - **July 2026:** Draft Shareholders Agreement with:
        - 4-year vesting schedule with 1-year cliff
        - Drag-along / Tag-along rights
        - Non-compete clause (2 years post-exit)

        - IP assignment (all code/algorithms owned by company, not individuals)
    - **August 2026:** File incorporation with Business Registration Service (BRS)

Equity Distribution (Proposed):

- Guandaru Kimathi: 55% (reflecting capital + operational leadership)
- Emmanuel Erogo: 35% (technical execution + co-founder status)
- ESOP Pool: 10% (reserved for future key hires: CTO, Head of Compliance, etc.)

---

3. CORE TECHNICAL BOTTLENECKS: THE SCALE-AWARENESS CRISIS

3.1 Current System Architecture Assessment

Stack: Django 4.2 + PostgreSQL 15 + Render (single dyno)
Capacity: Optimized for ~100 concurrent users
Critical Failure Point: System will crash under mass concurrent traffic (1,000+ simultaneous checkouts)

3.2 Identified Systemic Weaknesses

Problem 1: Database Performance Blindness

python

Current state: No indexes on frequently queried fields

Result: Sequential disk scans taking 2-5 seconds per query

Example bottleneck:

Event.objects.filter(date__gte=timezone.now()).order_by('date')

This query scans ENTIRE events table on every homepage load

Problem 2: Synchronous Payment Deadlocks

python

Current checkout flow (13-second blocking request):

@app.route('/checkout', methods=['POST'])

def checkout(request):

```
    # Step 1: Validate payment (waits for M-Pesa callback) → 5 seconds
    payment = mpesa.stk_push(phone, amount)

    # Step 2: Generate QR code → 1 second
    qr_code = generate_qr(ticket_id)

    # Step 3: Send email → 4 seconds
    send_email(user.email, qr_code)

    # Step 4: Send SMS → 3 seconds
    send_sms(user.phone, qr_code)

    return JsonResponse({'status': 'success'})  # User waits 13 seconds

# Critical Failure Modes:
# - User refreshes page → duplicate charge
# - Server timeout → payment deducted but no ticket generated
# - 500 concurrent users → server crashes
```

Problem 3: Zero Gate Verification Infrastructure

Current state: QR codes exist but no scanning mechanism

Result: Security guards manually checking ticket IDs (fraud-prone, slow)

Required: Mobile scanning app with <200ms verification time

Problem 4: Merchandise Race Conditions

python

```
# Inventory overselling scenario:
# User A and User B both buy last t-shirt simultaneously
```

Both transactions commit to database

Result: -1 inventory (oversold)

Root cause: No database-level locking or atomic transactions

---

4. DEVELOPMENT ROADMAP: THE API-FIRST TRANSFORMATION

4.1 Architectural Paradigm Shift

From: Monolithic Django templates (HTML rendered server-side)
To: Headless API Backend (JSON endpoints) + Decoupled Frontend (Next.js PWA)

Why This Matters:

- **Scalability:** API can serve mobile app, web app, scanner app, bank integrations from single codebase
- **Performance:** Stateless JSON responses are 10x faster than HTML template rendering
- **Testability:** API endpoints are trivially unit-testable (vs. integration testing full page renders)
- **Future-Proofing:** When we build LegalBeat (IP ledger), it plugs into same API infrastructure

Mandate: Every feature written from **May 18, 2026 onward** must be a **stateless JSON API endpoint** following RESTful conventions.

4.2 YOUR SIX DEVELOPMENT MANDATES (Q2 2026)

MANDATE 1: GATE VERIFICATION API (CRITICAL PATH)

Objective: Build universal ticket scanning infrastructure

Technical Specification:

python

Endpoint: POST /api/v1/gate/verify-ticket/

```
# Purpose: Authenticate QR code in <200ms
# Auth: Bearer token (scanner app gets unique API key)

@api_view(['POST'])
@permission_classes([IsAuthenticated])
def verify_ticket(request):
    """
    Input: {"qr_code": "abc123xyz", "gate_id": "main_entrance"}
    Output: {"valid": true, "ticket_id": "T-12345", "holder_name": "John Doe"}

    Performance: Must respond in <200ms (99th percentile)
    Security: HMAC verification to prevent counterfeit QRs
    """
    qr_code = request.data.get('qr_code')

    # Step 1: Decode QR (extract ticket_id + HMAC signature)
    ticket_id, signature = decode_qr(qr_code)

    # Step 2: Verify HMAC (prevent counterfeiting)
    if not verify_hmac(ticket_id, signature):
        return Response({"valid": False, "reason": "Invalid signature"}, status=403)

    # Step 3: Check ticket status (single database query, indexed)
    try:
        ticket = Ticket.objects.select_related('event', 'buyer').get(
            id=ticket_id,
```

```
        status='VALID'  # Not already scanned or refunded
    )
except Ticket.DoesNotExist:
    return Response({"valid": False, "reason": "Ticket not found"}, status=404)

# Step 4: Mark as scanned (atomic update)
ticket.status = 'SCANNED'
ticket.scanned_at = timezone.now()
ticket.scanned_by = request.user  # Security guard's account
ticket.save()

# Step 5: Return ticket details
return Response({
    "valid": True,
    "ticket_id": ticket.id,
    "holder_name": ticket.buyer.name,
    "event_name": ticket.event.title,
    "ticket_type": ticket.category
}, status=200)
```

Deliverables:

1. API endpoint implementation (views/gate_verification.py)
2. HMAC verification utility (utils/crypto.py)
3. Database migration adding scanned_at, scanned_by fields to Ticket model
4. API documentation (OpenAPI/Swagger spec)
5. **Postman collection** for testing
6. Unit tests achieving >90% code coverage

Success Criteria:

- P99 latency <200ms (tested with 1,000 concurrent requests via locust)
- 100% rejection of counterfeit QR codes (tested with manipulated signatures)
- Atomic scans (no double-scanning race conditions)

Deadline: June 7, 2026 (Sprint 1 close)

MANDATE 2: MERCHANDISE MODULE ISOLATION (DATA INTEGRITY)

Objective: Prevent inventory race conditions during high-concurrency checkouts

Current Problem:

python

```
# Overselling bug:
merchandise = Merchandise.objects.get(id=123)  # Stock: 1
if merchandise.stock >= 1:
    # User A and User B both pass this check simultaneously
    merchandise.stock -= 1  # Both subtract, stock becomes -1
    merchandise.save()  # Oversold!
```

Required Fix:

python

```
# Solution 1: Database-level locking
from django.db import transaction

@transaction.atomic
def purchase_merchandise(merch_id, quantity):
    # SELECT ... FOR UPDATE locks the row
    merch = Merchandise.objects.select_for_update().get(id=merch_id)
```

```
    if merch.stock < quantity:
        raise InsufficientStock("Only {} items remaining".format(merch.stock))

    merch.stock -= quantity
    merch.save()

    return merch

# Solution 2: Database constraint (PostgreSQL)
# ALTER TABLE merchandise ADD CONSTRAINT stock_non_negative CHECK (stock >= 0);
```

Technical Specification:

1. **Database Migrations:**
    - Add CHECK constraint on stock field (prevents negative values)
    - Add composite index on (vendor_id, status) for vendor dashboard queries
2. **API Endpoints:**

```
python
  POST /api/v1/merchandise/purchase/
  GET  /api/v1/merchandise/availability/{id}/
  GET  /api/v1/vendors/{id}/inventory/
```

3. **Inventory Management:**
    - Real-time stock updates via Redis cache
    - Pessimistic locking during checkout
    - Admin dashboard showing low-stock alerts

Deliverables:

1. Isolated merchandise models in models/merchandise.py
2. Transaction-safe purchase logic

3. Vendor inventory API
4. Admin panel enhancements
5. Stress test demonstrating no overselling under 500 concurrent purchases

Deadline: June 14, 2026 (Sprint 2 close)

MANDATE 3: AUTOMATED TESTING INFRASTRUCTURE (QUALITY ASSURANCE)

Objective: Eliminate manual QA bottleneck via comprehensive test suite

Current State: Zero automated tests → every deployment risks breaking production

Required Coverage:

python

```
# Test categories:
tests/
├── unit/                # Isolated function tests
│   ├── test_models.py     # Database logic
│   ├── test_serializers.py # Data validation
│   └── test_utils.py      # Crypto, helpers
├── integration/         # Multi-component workflows
│   ├── test_checkout.py   # Full purchase flow
│   ├── test_webhooks.py   # M-Pesa callbacks
│   └── test_auth.py       # Login/registration
└── performance/         # Load testing
    └── locustfile.py      # 1000+ concurrent users
```

Example Test (Checkout Flow):

python

```
import pytest
from django.test import Client
```

```
from decimal import Decimal


@pytest.mark.django_db
class TestCheckoutFlow:
    def test_successful_ticket_purchase(self):
        """
        Verify complete checkout workflow:
        1. User selects ticket
        2. Initiates M-Pesa payment
        3. Receives QR code
        4. Email/SMS sent
        """
        client = Client()

        # Setup
        event = Event.objects.create(title="Test Concert", price=Decimal("2000.00"))
        user = User.objects.create_user(username="test", password="test123")
        client.login(username="test", password="test123")

        # Execute checkout
        response = client.post('/api/v1/checkout/', {
            "event_id": event.id,
            "ticket_quantity": 2,
            "phone_number": "254793595290"
        })
```

```
    # Assertions
    assert response.status_code == 200
    assert Ticket.objects.filter(event=event, buyer=user).count() == 2
    assert response.json()['qr_code'] is not None

  def test_insufficient_tickets(self):
    """
    Verify system rejects purchase when tickets sold out
    """
    event = Event.objects.create(
      title="Sold Out Show",
      total_tickets=10,
      tickets_sold=10  # Already sold out
    )

    response = client.post('/api/v1/checkout/', {
      "event_id": event.id,
      "ticket_quantity": 1
    })

    assert response.status_code == 400
    assert "sold out" in response.json()['error'].lower()
```

Implementation Stack:

- **Framework:** pytest-django (industry standard)
- **Coverage Tool:** coverage.py (enforce >85% coverage)
- **CI/CD:** GitHub Actions (run tests on every commit)

- **Load Testing:** locust (simulate 1000+ concurrent users)

Deliverables:

1. 50+ unit tests covering critical models and utilities
2. 20+ integration tests covering user workflows
3. Load test simulating 1,000 concurrent checkouts
4. GitHub Actions workflow (.github/workflows/tests.yml)
5. Coverage report achieving >85% codebase coverage

Deadline: June 21, 2026 (Sprint 3 close)

MANDATE 4: REDIS CACHING LAYER (PERFORMANCE OPTIMIZATION)

Objective: Reduce database load by 90% via intelligent caching

Problem:

python

Homepage query (currently runs on EVERY page load):

Event.objects.filter(date__gte=timezone.now()).order_by('date')[:10]

If 1,000 users visit homepage simultaneously:

- 1,000 database queries

- 5 seconds per query

- Database crashes

Solution:

python

Cache the result for 5 minutes

from django.core.cache import cache

def get_upcoming_events():

```
    cache_key = 'homepage:upcoming_events'
    events = cache.get(cache_key)

    if events is None:
        # Cache miss → query database
        events = list(Event.objects.filter(
            date__gte=timezone.now()
        ).order_by('date')[:10])

        # Store in cache for 5 minutes
        cache.set(cache_key, events, timeout=300)

    return events

# Result:
# - First user: 5 second database query
# - Next 999 users: 5 millisecond cache hit
# - 1000x performance improvement
```

Implementation Scope:

High-Value Cache Targets:

1. **Event Listings:** Homepage, search results, category pages
2. **LPS Scores:** Expensive to calculate (3+ API calls), cache for 1 hour
3. **User Profiles:** Rarely change, cache for 10 minutes
4. **Static Content:** Terms of Service, FAQ (cache for 24 hours)

Technical Stack:

- **Cache Backend:** Redis (in-memory key-value store)

- **Django Integration:** django-redis
- **Hosting:** Render Redis addon or Upstash (serverless Redis)

Cache Invalidation Strategy:

```
python

# Invalidate cache when data changes

from django.db.models.signals import post_save

from django.dispatch import receiver


@receiver(post_save, sender=Event)

def invalidate_event_cache(sender, instance, **kwargs):

    """

    When event is created/updated, clear related caches

    """

    cache.delete('homepage:upcoming_events')

    cache.delete(f'event:{instance.id}:details')
```

Deliverables:

1. Redis configuration in settings.py
2. Cache decorators for high-traffic views
3. Cache invalidation signals
4. Performance benchmark (before/after comparison)
5. Documentation on cache architecture

Success Criteria:

- Homepage load time: 5 seconds → 500ms (10x improvement)
- Database query count: 1,000/min → 100/min (90% reduction)
- Redis hit rate: >80% (measured via redis-cli INFO stats)

Deadline: June 28, 2026 (Sprint 4 close)

MANDATE 5: LEGAL COMPLIANCE LAYER (TERMS ENFORCEMENT)

Objective: Ensure every transaction has explicit legal consent

Current Risk:

User buys ticket → No Terms of Service acceptance

→ Disputes arise → We have no legal proof of agreement

→ Refund forced even for fraudulent claims

Required Implementation:

1. Database Schema:

```
python

class LegalConsent(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    document_type = models.CharField(max_length=50)  # 'TOS', 'PRIVACY', 'REFUND'
    document_version = models.CharField(max_length=20)  # 'v2.1'
    consented_at = models.DateTimeField(auto_now_add=True)
    ip_address = models.GenericIPAddressField()
    user_agent = models.TextField()  # Browser fingerprint

    class Meta:
        indexes = [
            models.Index(fields=['user', 'document_type']),
        ]
```

2. Checkout Validation:

```
python

@api_view(['POST'])
def checkout(request):
```

```
    # Mandatory consent check
    if not request.data.get('accept_terms'):
        return Response({
            "error": "You must accept Terms of Service to proceed",
            "terms_url": "https://zozaprime.com/legal/terms"
        }, status=400)

    # Log consent with timestamp + IP
    LegalConsent.objects.create(
        user=request.user,
        document_type='TOS',
        document_version='v2.1',
        ip_address=request.META.get('REMOTE_ADDR'),
        user_agent=request.META.get('HTTP_USER_AGENT')
    )

    # Proceed with checkout...
```

3. Frontend Enforcement:

```
javascript
// Checkout form (required checkbox)
<form onSubmit={handleCheckout}>
  <label>
    <input
      type="checkbox"
      required
      onChange={(e) => setAcceptTerms(e.target.checked)}
```

```
    />
    I accept the <a href="/terms">Terms of Service</a> and
    <a href="/refund-policy">Refund Policy</a>
  </label>

  <button disabled={!acceptTerms}>Complete Purchase</button>
</form>
```

Deliverables:

1. LegalConsent model with migrations
2. API validation middleware
3. Admin dashboard showing consent logs
4. Legal document versioning system
5. Audit trail export (for legal disputes)

Deadline: July 5, 2026 (Sprint 5 close)

MANDATE 6: STATELESS PASSWORD RECOVERY (ASYNC SECURITY)

Objective: Bulletproof password reset without blocking web server

Current Problem:

```
python
# Synchronous email sending (blocks for 4 seconds)
@app.route('/forgot-password', methods=['POST'])
def forgot_password(request):
    user = User.objects.get(email=request.data['email'])

    # Generate reset token
    token = generate_reset_token(user)
```

```
    # Send email (BLOCKS for 4 seconds while SMTP connects)
    send_email(user.email, f"Reset link: {token}")

    return Response({"message": "Email sent"})

# Problem: Server can only handle 1 password reset per 4 seconds
```

Solution: Background Task Queue

```
python
# views.py (instant response)
@app.route('/forgot-password', methods=['POST'])
def forgot_password(request):
    email = request.data['email']

    # Queue background task (returns immediately)
    send_password_reset_email.delay(email)

    # User sees instant response
    return Response({
        "message": "If that email exists, you'll receive reset instructions shortly"
    }, status=200)

# tasks.py (runs in background worker)
from celery import shared_task

@shared_task
```

```
def send_password_reset_email(email):
    try:
        user = User.objects.get(email=email)
        token = generate_reset_token(user)

        # Send email (takes 4 seconds but doesn't block web server)
        send_email(
            to=email,
            subject="Password Reset Request",
            body=f"Click here to reset: https://zozaprime.com/reset/{token}"
        )
    except User.DoesNotExist:
        # Don't reveal whether email exists (security best practice)
        pass
```

Security Requirements:

1. **Token Generation:**

```
python
  import secrets
  from datetime import timedelta

  def generate_reset_token(user):
      # Cryptographically secure random token
      token = secrets.token_urlsafe(32)  # 256 bits entropy

      # Store in database with expiry
      PasswordReset.objects.create(
```

```
        user=user,
        token=token,
        expires_at=timezone.now() + timedelta(hours=1)  # 1-hour expiry
    )

    return token
```

2. **Token Validation:**

```
python
@app.route('/reset-password', methods=['POST'])
def reset_password(request):
    token = request.data['token']
    new_password = request.data['new_password']

    # Verify token exists and hasn't expired
    try:
        reset = PasswordReset.objects.get(
            token=token,
            expires_at__gte=timezone.now(),
            used=False
        )
    except PasswordReset.DoesNotExist:
        return Response({"error": "Invalid or expired token"}, status=400)

    # Update password
    user = reset.user
    user.set_password(new_password)
```

```
    user.save()

    # Mark token as used (prevent reuse)
    reset.used = True
    reset.save()

    return Response({"message": "Password reset successful"})
```

Deliverables:

1. Celery task queue configuration
2. PasswordReset model with expiry logic
3. Email templates (HTML + plain text)
4. Rate limiting (max 3 resets per hour per email)
5. Security audit documentation

Deadline: July 12, 2026 (Sprint 6 close)

---

5. SPRINT TIMELINE & ACCOUNTABILITY

Sprint	Dates	Deliverable	Status Check
Sprint 1	May 18 - Jun 7	Gate Verification API	Jun 7 (Sunday 8pm sync call)
Sprint 2	Jun 8 - Jun 14	Merchandise Isolation	Jun 14 (code review + demo)
Sprint 3	Jun 15 - Jun 21	Testing Infrastructure	Jun 21 (coverage report)
Sprint 4	Jun 22 - Jun 28	Redis Caching	Jun 28 (performance benchmark)
Sprint 5	Jun 29 - Jul 5	Legal Compliance	Jul 5 (audit trail demo)
Sprint 6	Jul 6 - Jul 12	Password Recovery	Jul 12 (final security review)

Definition of Done (Each Sprint):

1. Code merged to develop branch

2. All tests passing (>85% coverage)
3. API documentation updated (Swagger)
4. Deployed to staging environment
5. Demo video recorded (Loom/WhatsApp)

---

6. COMMUNICATION & DELIVERABLE PROTOCOL

GitHub Workflow:

main (production) ← Never commit directly

↑

develop (staging) ← Merge from feature branches after review

↑

feature/gate-verification ← Your working branch

feature/merchandise-isolation

feature/testing-suite

Commit Message Standard:

[MANDATE-1] Add gate verification endpoint

- Implement POST /api/v1/gate/verify-ticket/

- Add HMAC signature validation

- Include unit tests (95% coverage)

- Update API documentation

Resolves #23

Code Review Process:

1. You push to feature branch
2. Create Pull Request with description

3. I review within 24 hours (async)
4. Address comments
5. Merge to develop

Blocker Escalation:

- **Minor:** Document in PR comments
- **Major:** WhatsApp message with "BLOCKER:" prefix
- **Critical:** Phone call (reserved for production outages only)

---

7. CAPITAL ACCOUNT ACTION ITEMS

Your Tasks (Deadline: May 31, 2026):

1. **Document Historical Contributions:**
    - Compile all receipts/invoices for domain registration, hosting fees, third-party services paid from your accounts
    - Format: Excel sheet with columns: Date | Amount | Description | Receipt Link
    - Send to: guandaru@zozaprime.com
2. **Confirm Equity Proposal:**
    - Review proposed equity split (55% Guandaru / 35% Emmanuel / 10% ESOP)
    - Provide counter-proposal if disagreement
    - Schedule equity negotiation call if needed
3. **Sign Founder Loan Agreement:**
    - Review draft agreement (I'll send by May 25)
    - Provide feedback by May 28
    - Digital signature by May 31

---

8. FINAL NOTES

Emmanuel, this restructuring positions us to scale from **fragile prototype** to **institutional-grade infrastructure**. Your execution on these six mandates is the technical foundation that enables everything else: bank partnerships, investor confidence, creator trust.

Key Mindset Shifts:

1. **From "Making It Work" → "Making It Scale"**
    - Every line of code now must handle 1,000+ concurrent users
    - Performance is not optional, it's existential
2. **From "Ship Fast" → "Ship Right"**
    - Automated tests eliminate 90% of QA bottleneck
    - Code review catches bugs before they reach users
3. **From "Solo Coding" → "API-First Architecture"**
    - Build once, use everywhere (web, mobile, scanner, banks)
    - Future LEGALBEAT integration depends on clean APIs today

Your agency over technical execution is absolute. I won't second-guess engineering decisions—that's your domain. My role is to ensure the commercial side (bank deals, events, grants) is ready when the tech ships.

Let's execute. Next sync: **Sunday, 31st May , 8:00 PM** (Sprint 1 review on Scanner App).

Guandaru Dedan Kimathi
Co-Founder & CEO
ZOZAPRIME Ltd.